

Simulating Spectral Sampling of the MICA Files with Enfys: InAs SNR10x MWIR

August 27, 2025

1 Abstract

Here we investigate the expected ability of a new remote-sensing stand-off infrared spectrometer, *Enfys*, for the ExoMars Rosalind Franklin Rover, to resolve diagnostic features of the reflectance spectra of key minerals previously identified on the surface of Mars, using the Spectral Angle as a metric.

In this notebook we simulate Enfys performance using an InAs Mid-Wave Infrared (1.6 - 3.1 μm , ‘MWIR’) sensor, that is subject to thermal noise from the Enfys structure.

Here we consider a worst case scenario where an SNR of 1/10th of the best case is achieved in the InAs channel.

2 Introduction

As discussed in [previous notebooks](#), Enfys is an Infra-Red point-spectrometer for the ExoMars Rosalind Franklin rover.

In this notebook, and the others accompanying it, we explore how we expect Enfys to sample Minerals Identified through CRISM Analysis, as defined by the ‘MICA’ files.

In this notebook, we consider the expected performance of Enfys using an InAs Mid-Wave Infrared sensor for the wavelength range of 1.6 - 3.1 μm . Included in the modelling is the noise expected for the sensor, as provided by the EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 document (contact Enfys P.I. M. Gunn).

2.1 Problem

The optical design of Enfys is functionally different to the Acousto-Optic Tunable Filter design of its predecessor ISEM, so it is important that the design is demonstrated to fulfil the performance requirements of the instrument, and that expectations of performance can be set prior to laboratory tests of the completed instrument.

At time of writing, the spectrometer is in the development stage. This notebook describes how the Spectral Parameters Toolkit¹ can be used to convert the developing expectations of the spectral range and resolution and radiometric sensitivity of the instrument into predictions of its ability to discriminate mineral species and phases on the Mars surface at the rover landing site.

¹Stabbins, R., & Grindrod, P. (2024). The Spectral Parameters Toolkit (sptk): Release v0.1 (Version v0.1) [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.10694286>

In these notebooks we consider two candidate MWIR sensors, that trade-off spectral range against Signal-to-Noise Ratio.

Candidate A is the InAs sensor, simulated here, that is sensitive up to $\sim 3.1 \mu\text{m}$, with a nominal SNR range of 100 - 600.

Candidate B is a modified InGaAs sensor, sensitive to $\sim 2.5 \mu\text{m}$, with a nominal SNR range of

Here we consider a worst-case-scenario where InAs sensor only achieves 1/10th of the nominal SNR. This is to approximate uncertainty in the thermal state of the instrument enclosure, that is the dominant source of thermal noise.

3 Method

To investigate the expected performance we use high-resolution laboratory measured spectra of a range of minerals expected at the Mars surface, and computationally simulate the sampling of the spectra with the expected spectral response of *Enfys*. We will perform repeat sampling with synthetically added noise to investigate the spectral and radiometric limits of the instrument.

We will produce a number of visualisations to investigate the predicted instrument performance, and demonstrate the limits of the instrument discriminatory ability.

We will quantify the ability of the instrument to resolve the defined minerals of interest by computing the Spectral Angle between a noisy sampled spectrum of a given mineral and all other ‘clean’ (resampled to *Enfys* spectral channels, but without noise added). We will compare the noisy sample Spectral Angle to a noise-free baseline Spectral Angle, to understand the performance limits and the impact of the simulated instrument noise.

3.1 Spectral Resampling

We take a simple approach to resampling, as follows.

We neglect reflectance calibration uncertainty, and any systematic uncertainty when merging the data from the two separate photodiodes, and assume that the linear-variable filter (LVF) movement is perfectly calibrated such that a measurement at centre-wavelength λ_{cwl} can be requested with arbitrary precision.

We assume an observation of λ_{cwl} has a Cauchy transmission profile. We use the latest expectations of the spectral resolving power of the short-wave infrared (SWIR) and mid-wave infrared (MWIR) LVFs, that we have linearly interpolated from ~ 30 measurements from $0.9 - 2.6 \mu\text{m}$ to arbitrary values of λ in the SWIR and MWIR ranges, to give the vector $\Gamma[\lambda]$. The Cauchy transmission profile for a given λ_{cwl} has a full-width-at-half-maximum of $\Delta\lambda = \lambda_{cwl}/\Gamma[\lambda_{cwl}]$.

This data has been digitised from the plot reported in the `Enfys schedule review 20250807.ppt`, by Matt Gunn.

```
[27]: import pandas as pd

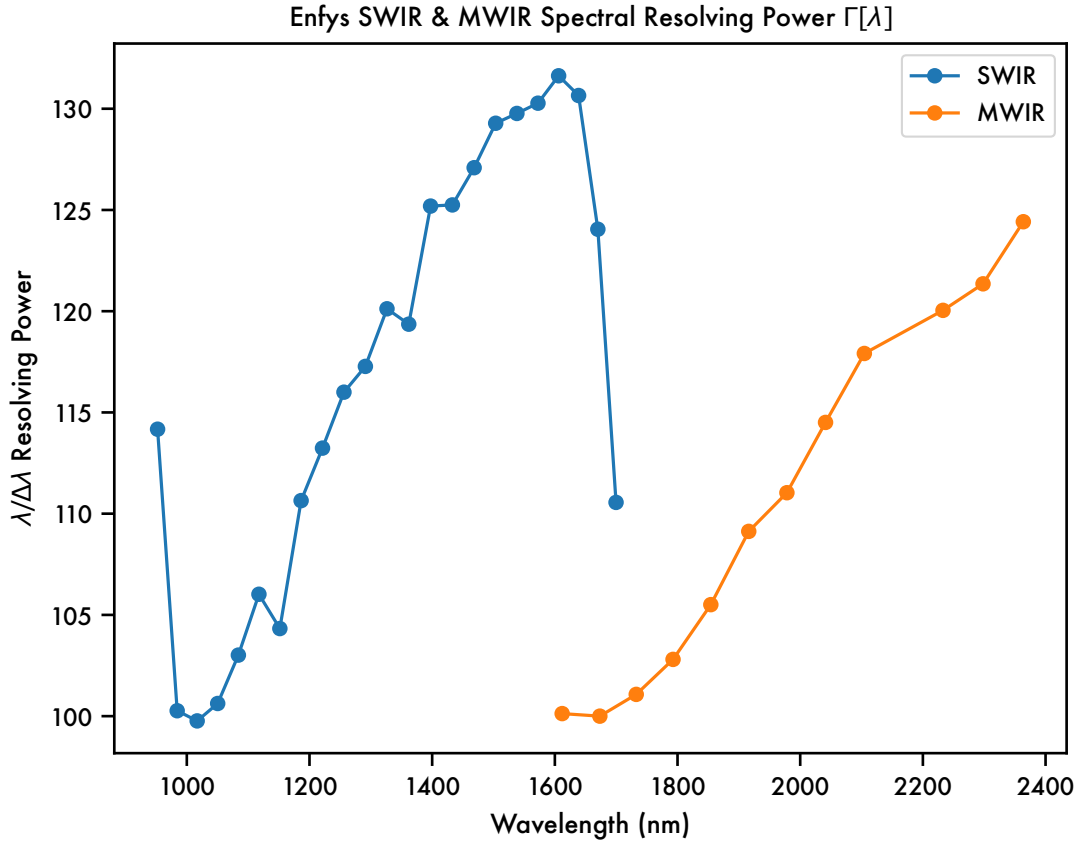
enfys_swir_respon = pd.read_csv('../data/instruments/enfys_swir_respon.csv',
                                index_col=0)
```

```

enfys_mwir_respow = pd.read_csv('../data/instruments/enfys_mwir_respow.csv',
    ↪index_col=0)
g=enfys_swir_respow.plot(label='SWIR', marker='o')
enfys_mwir_respow.plot(marker='o',label='MWIR', xlabel='Wavelength (nm)',
    ↪ylabel=r'$\lambda/\Delta\lambda$ Resolving Power',ax=g)
g.legend(['SWIR', 'MWIR'])
g.set_title('Enfys SWIR & MWIR Spectral Resolving Power $\Gamma[\lambda]$')

```

[27]: Text(0.5, 1.0, 'Enfys SWIR & MWIR Spectral Resolving Power \$\Gamma[\lambda]\$')



The Cauchy transmission profile $T(\lambda|\lambda_{cwl})$ is given by

$$T(\lambda|\lambda_{cwl}) = \frac{\gamma^2}{(\lambda - \lambda_{cwl})^2 + \gamma^2}$$

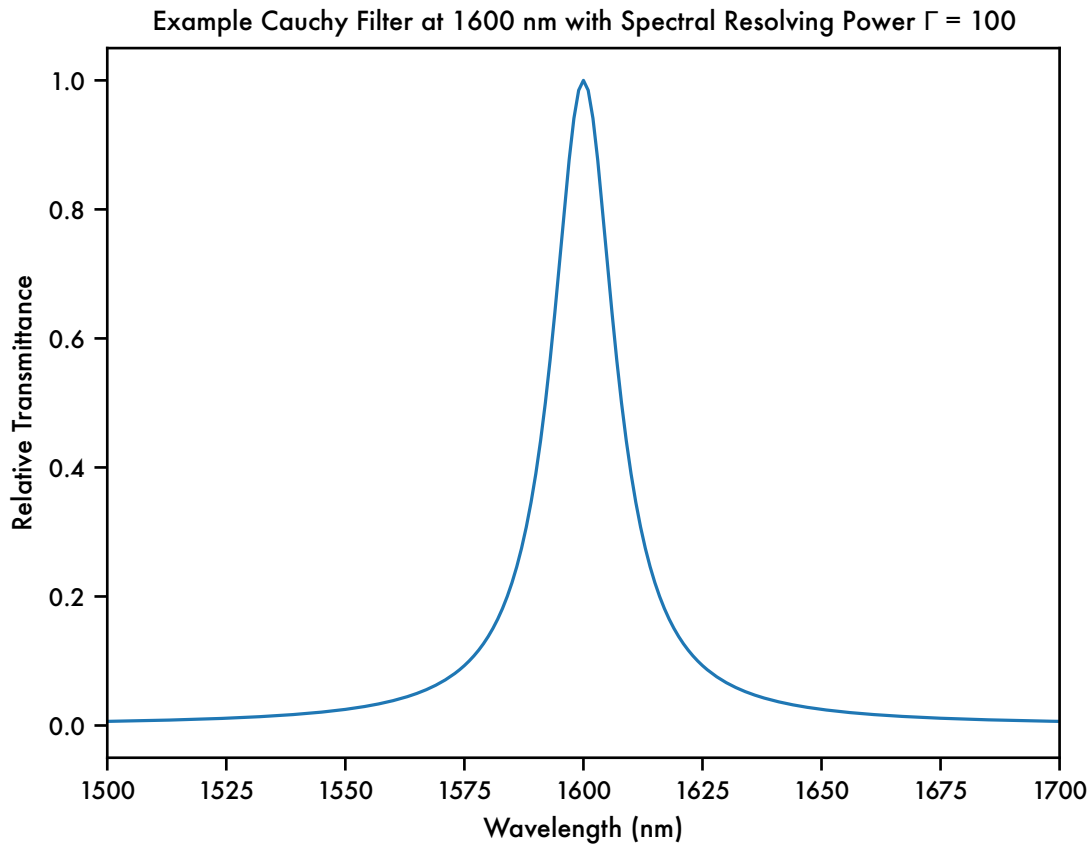
where $\gamma = \Delta\lambda/2 = (\lambda_{cwl}/\Gamma[\lambda_{cwl}])/2$.

The shape of the Cauchy profile is illustrated here.

```
[55]: from sptk.instrument import Instrument
import sptk.config as cfg
from matplotlib import pyplot as plt

exmpl_cauchy = Instrument.build_cauchy_filter(1600, 1600/100)
plt.plot(cfg.WVLS, exmpl_cauchy.squeeze())
plt.xlim(1500, 1700)
plt.title('Example Cauchy Filter at 1600 nm with Spectral Resolving Power_
↪$\Gamma$ = 100')
plt.xlabel('Wavelength (nm)')
plt.ylabel('Relative Transmittance')
```

```
[55]: Text(0, 0.5, 'Relative Transmittance')
```



We assume that a high-resolution laboratory spectrum approximates the continuous function $R(\lambda)$, such that the calibrated reflectance at λ_{cwl} has the expected value $R[\lambda_{cwl}]$ of:

$$R[\lambda_{cwl}] = \frac{\int R(\lambda)T(\lambda|\lambda_{cwl})d\lambda}{\int T(\lambda|\lambda_{cwl})d\lambda}$$

We use this equation to compute the Enfys-sampled reflectance spectra.

3.2 Adding Noise

To simulate noise we add ε to a reflectance measurement, an additive noise term drawn from the Gaussian distribution with standard deviation σ , that we define relative to the specified Signal-to-Noise Ratio (SNR),

$$\sigma = \frac{S}{\text{SNR}}$$

where S is the signal.

The supplied Signal-to-Noise Ratio vector has been defined under the assumption of a 30% albedo spectrally uniform reflector.

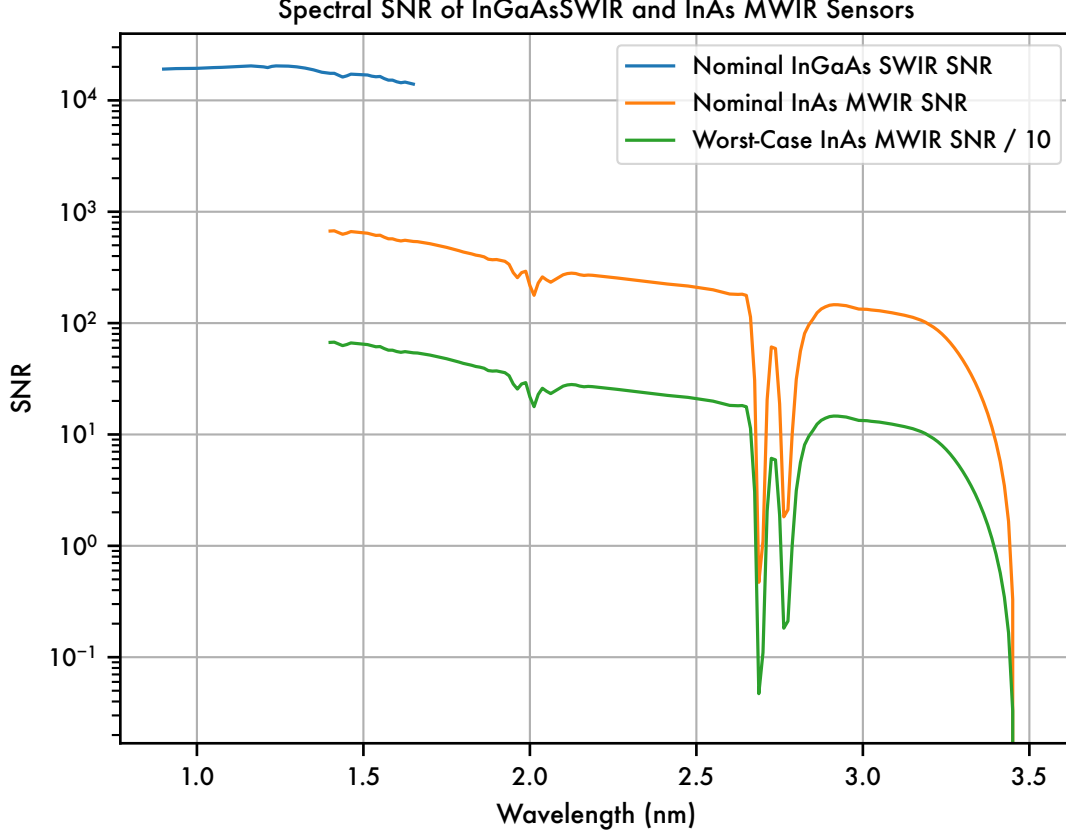
The expected spectral SNR of the SWIR and MWIR sensors has been measured and processed by P.I. M. Gunn, and reported in the EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 spreadsheet.

We read in the SNR file, and get the SNR data for the InAs MWIR sensor and the InGaAs SWIR sensor,

```
[74]: enfys_snr = pd.read_csv('../data/instruments/enfys_snr.csv', index_col=0)
      inas_mwir_snr = enfys_snr['InAs MWIR SNR']
      ingaas_swir_snr = enfys_snr['InGaAs SWIR SNR']
```

The spectral SNR for each of the SWIR InGaAs and MWIR InAs sensors are:

```
[78]: g = ingaas_swir_snr.plot(grid=True, label='Nominal InGaAs SWIR SNR')
      inas_mwir_snr.plot(logy=True, grid=True, label='Nominal InAs MWIR SNR')
      (inas_mwir_snr/10).plot(logy=True, grid=True, label='Worst-Case InAs MWIR SNR / 10')
      g.set_xlabel('Wavelength (nm)')
      g.set_ylabel('SNR')
      g.legend()
      title = g.set_title('Spectral SNR of InGaAsSWIR and InAs MWIR Sensors')
```



We assume that this SNR vector is modulated by the reflectance of the mineral of interest, such that σ as a function of the spectral Signal-to-Noise Ratio $\text{SNR}[\lambda]$ is:

$$\sigma[\lambda] = \frac{R[\lambda]}{\text{SNR}[\lambda]}$$

For each observation $R[\lambda_{cwl}]$ we draw n -repeat noisy samples $\tilde{R}_i[\lambda_{cwl}]$, such that

$$\tilde{R}_i[\lambda_{cwl}] = R[\lambda_{cwl}] + \varepsilon_i[\lambda_{cwl}]$$

with

$$\varepsilon_i[\lambda_{cwl}] \sim \mathcal{N}(0, \sigma[\lambda_{cwl}])$$

This gives an indication of the expected noise of a calibrated *Enfys* acquired spectrum.

3.3 Analysis

First we produce plots of the resampled spectra, and single instances of noisy spectra, and visually inspect these to consider the ability of this *Enfys* configuration to resolve diagnostic spectral features

against the noise.

3.3.1 Spectral Angle

We then take a single noisy sample and evaluate the Spectral Angle between the noisy sample and each noise-free entry of the MICA files spectral database that we are sampling. We repeat this for each noisy sample, to give the mean Spectral Angle and it's uncertainty, to determine the ability to dicriminate between similar mineral reflectance spectra.

The Spectral Angle is defined between the noisy spectrum $\tilde{R}_i[\lambda]$ of material i in the set of MICA file materials ($\mathcal{M}$) and the clean spectrum $R_j[\lambda]$, where $j \in \mathcal{M}$.

The Spectral Angle θ is:

$$\theta(i, j) = \arccos \left(\frac{\sum_{\lambda} \tilde{R}_i[\lambda] R_j[\lambda]}{(\sum_{\lambda} \tilde{R}_i[\lambda]^2)^{1/2} (\sum_{\lambda} R_j[\lambda]^2)^{1/2}} \right)$$

3.4 Notebook and Environment Setup

Here we include the required code blocks for setting up this notebook for this study.

```
[ ]: %load_ext autoreload
      %autoreload 2
      %config InlineBackend.figure_format = 'svg' # note - requires Inkscape to be
      ↪ installed, and to be accessible from the terminal
      # on macos, add inkscape to the path with `sudo ln -s /Applications/Inkscape.
      ↪ app/Contents/MacOS/inkscape /usr/local/bin/inkscape`
      %matplotlib inline
```

We will be using the Spectral Parameters Toolkit. First we import the required modules.

```
[ ]: import sptk.config as cfg
      from sptk.material_collection import MaterialCollection
      from sptk.instrument import Instrument
      from sptk.observation import Observation
      from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla
```

We set our simulation resolution range to extend the range of Enfys, of 0.9 - 3.1 μm , such that the long tails of the Cauchy distribution of the linear-variable-filter profiles can be accommodated during sampling. We leave our spectral resolution at 1 nm for all λ .

```
[ ]: cfg.update_sample_res('wvl_min', 380)
      cfg.update_sample_res('wvl_max', 3300)
```

Finally we set the project name.

```
[ ]: PROJECT_NAME = 'enfys_mica_inas_mwir_snr10x'
```

4 Data

For this study we are using a spectral library of minerals identified through CRISM analysis ([the MICA files](#))². Here we use the laboratory spectra that have been identified that best represent the type localities presented in the MICA files. These can be accessed [here](#), and are sourced from the RELAB and USGS public spectral libraries.

We have grouped the spectral library into categories as follows, and included this categorisation in the `sptk` distribution, accessed from the `sptk.material_sets` module.

```
[ ]: from sptk.material_sets import MICA_SET
```

We use the `sptk.material_collection` module to parse the spectral library:

```
[ ]: matcol = MaterialCollection(  
    MICA_SET,  
    'MICA_new_dirtree',  
    PROJECT_NAME,  
    load_existing=False,  
    balance_classes=False,  
    random_bias_seed=None,  
    allow_out_of_bounds=True,  
    plot_profiles=False,  
    export_df=True)
```

Building new enfys_mica_inas_mwir_snr10x MaterialCollection DF

Category

species|subgroup|group|
data_id status

Iron Oxides & Primary Silicates

-clinopyroxene|pyroxenes|inosilicates|
-NBPP22.csv loaded
-orthopyroxene|pyroxenes|inosilicates|
-LAPP47B.csv loaded
-fayalite|olivines|nesosilicates|
-LAP005.csv loaded
-forsterite|olivines|nesosilicates|
-LASC02.csv loaded
-hematite|hematites|oxides|
-GDS27.csv loaded
-plagioclase|feldspars|tectosilicates|
-NCLS04.csv loaded

²Viviano-Beck, C. E., Seelos, F. P., Murchie, S. L., Kahn, E. G., Seelos, K. D., Taylor, H. W., Taylor, K., Ehlmann, B. L., Wiseman, S. M., Mustard, J. F., & Morgan, M. F. (2014). Revised CRISM spectral parameters and summary products based on the currently detected mineral diversity on Mars. *Journal of Geophysical Research: Planets*, 119(6), 1403–1431. <https://doi.org/10.1002/2014JE004627>

Sulfates

- alunite|hydrous_sulphates|sulphates|
-F1CC08B.csv loaded
- bassanite|hydrous_sulphates|sulphates|
-GDS145.csv loaded
- gypsum|hydrous_sulphates|sulphates|
-LASF41A.csv loaded
- jarosite|hydrous_sulphates|sulphates|
-LASF21A.csv loaded
- kieserite|hydrous_sulphates|sulphates|
-F1CC15.csv loaded
- mg-sulphate|hydrous_sulphates|sulphates|
-799F366.csv loaded

Phyllosilicates

- chlorite|chlorites|phyllosilicates|
-LACL14.csv loaded
- kaolinite|kaolinites|phyllosilicates|
-LAKA04.csv loaded
- illite|micas|phyllosilicates|
-LAIL02.csv loaded
- margarite|micas|phyllosilicates|
-GDS106.csv loaded
- vermiculite|micas|phyllosilicates|
-c1cy29.csv loaded
- serpentine|serpentine|phyllosilicates|
-LALZ01.csv loaded
- montmorillonite|smectites|phyllosilicates|
-LAM002.csv loaded
- nontronite|smectites|phyllosilicates|
-NCJB26.csv loaded
- saponite|smectites|phyllosilicates|
-LASA58.csv loaded
- talc|talcs|phyllosilicates|
-GDS23.csv loaded

Carbonates

- calcite|calcites|carbonates|
-CAGR01.csv loaded
- magnesite|calcites|carbonates|
-F1CC06B.csv loaded

Hydrated Silicates & Halides

- halite|chlorides|halides|
-HS433.csv loaded
- prehnite|prehnites|inosilicates|
-LAZE03.csv loaded

```
-siliceous-sinter|hydrated-silicas|mineraloids|  
  -bkr1jb329c.csv loaded  
-epidote|epidotes|sorosilicates|  
  -GDS26.csv loaded  
-analcime|zeolites|tectosilicates|  
  -GDS1.csv loaded  
Loading 29 of entries complete.  
Exporting the Material Collection to CSV and Pickle formats...  
We can use the material_collection plotting routines to visualise the library.
```

```
[ ]: ax = matcol.plot_profiles(stacked=True, scope='categories', groupby='Species')
```

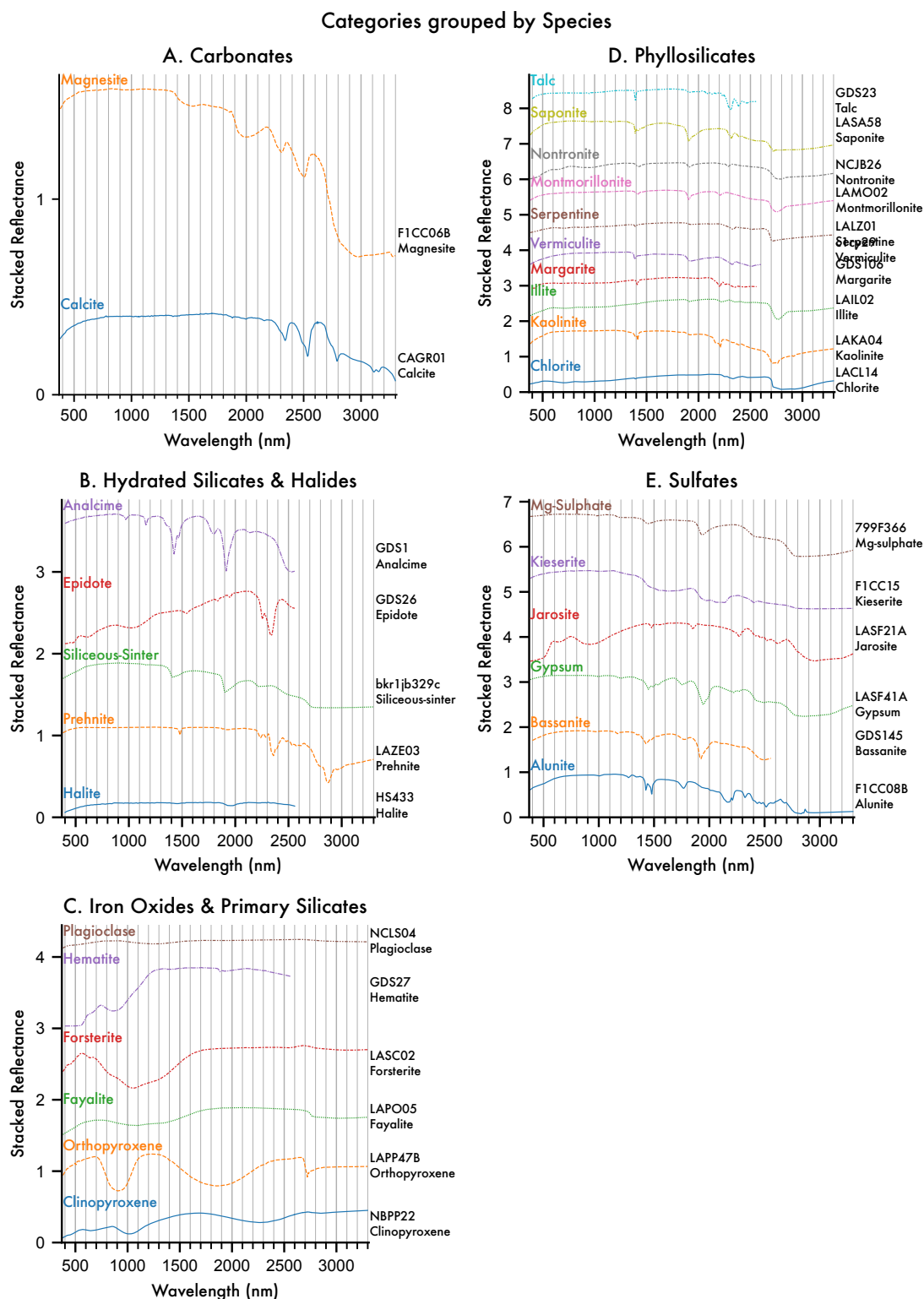


Figure 2 High resolution MICA Files spectral library, arranged by category and grouped by mineral

species.

We can see that not all of the entries cover the complete 0.9 - 3.1 μm range of *Enfys*, so in future work we should look to replace these MICA files laboratory type spectra with spectra that all span the entire range.

We can also produce a waterfall visualisation, an alternative plotting method that presents the information in a condensed format, that is useful for highlighting where absorption features lie.

```
[ ]: axes = matcol.plot_profiles(waterfall=True, scope='all', groupby='Category')
```

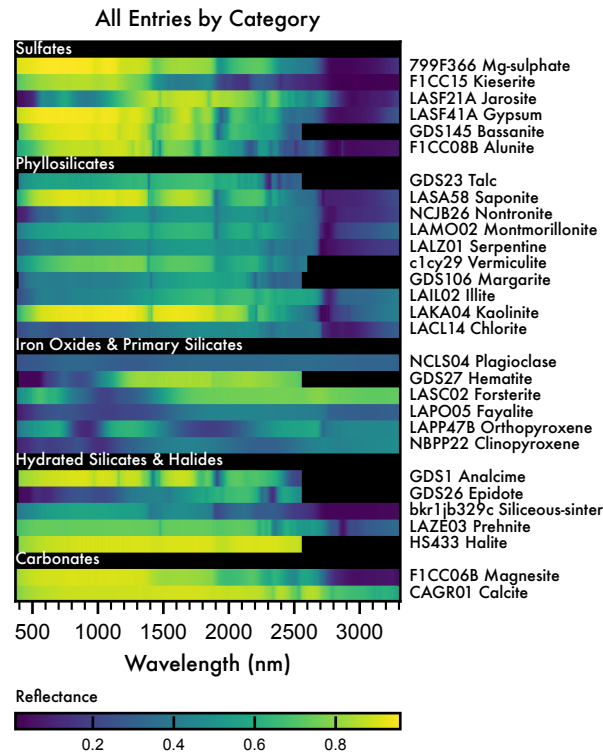


Figure 3 Waterfall plot of the high resolution MICA Files spectral library, arranged by category and grouped by mineral species.

A more useful version of this plot format is to show band locations after continuum removal. We can use the `SpectralLibraryAnalyser.remove_continuum()` function to produce a duplicate of the `MaterialCollection` but with the data continuum-removed.

```
[ ]: from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

matcol_sla = sla(matcol)
matcol_cr = matcol_sla.remove_continuum()
```

We can plot the profiles to illustrate the continuum removal. Here we use the `scope = 'all'` version of the profile plot.

```
[ ]: axes = matcol_cr.plot_profiles(stacked=True, scope='categories',
    ↳groupby='Species')
```

Categories grouped by Species
Continuum Removed

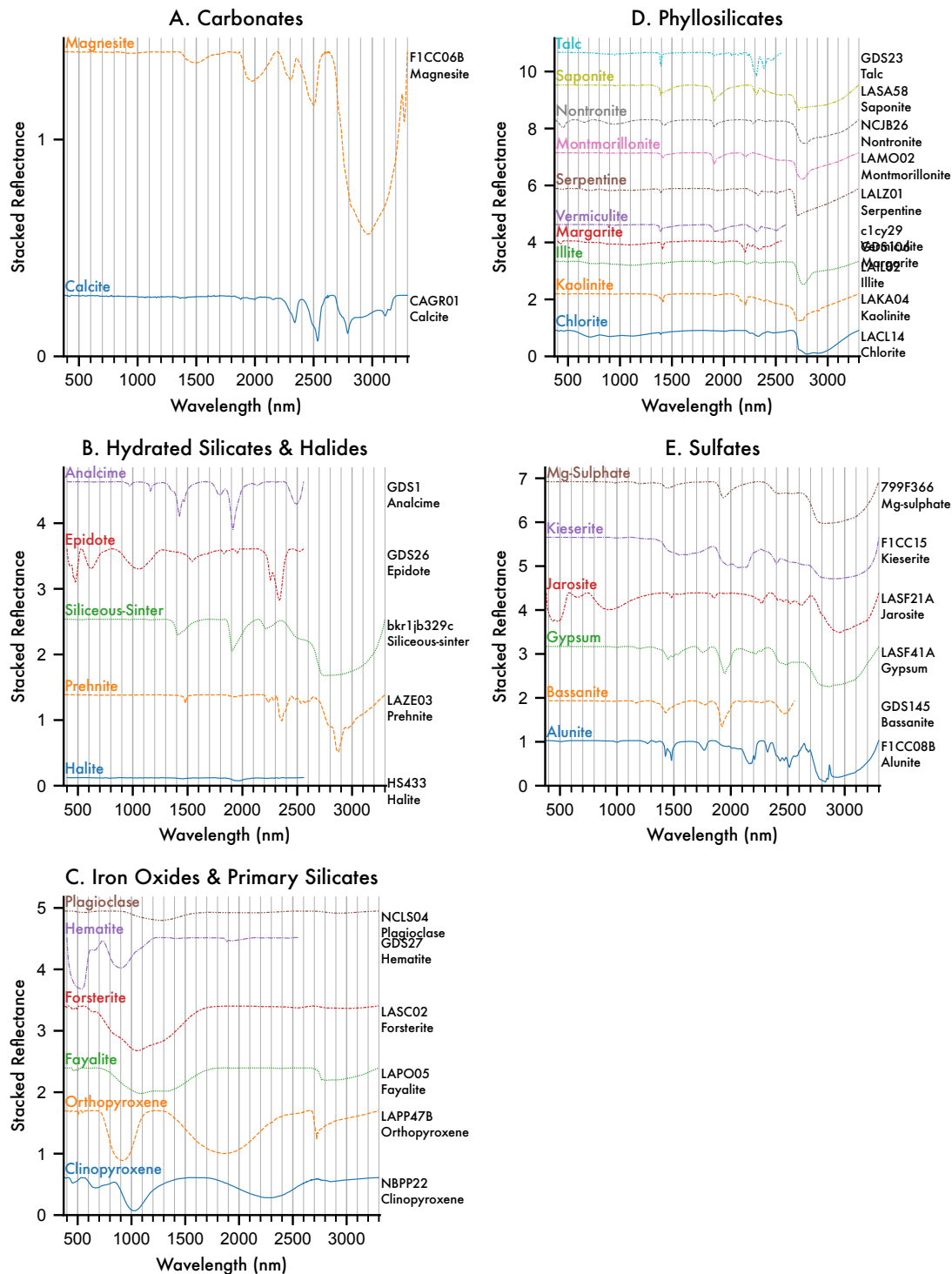


Figure 4 Continuum-removed high resolution MICA Files spectral library, grouped by Category.

Now we plot the waterfall representation:

```
[ ]: axes = matcol_cr.plot_profiles(waterfall=True, scope='all', groupby='Category')
```

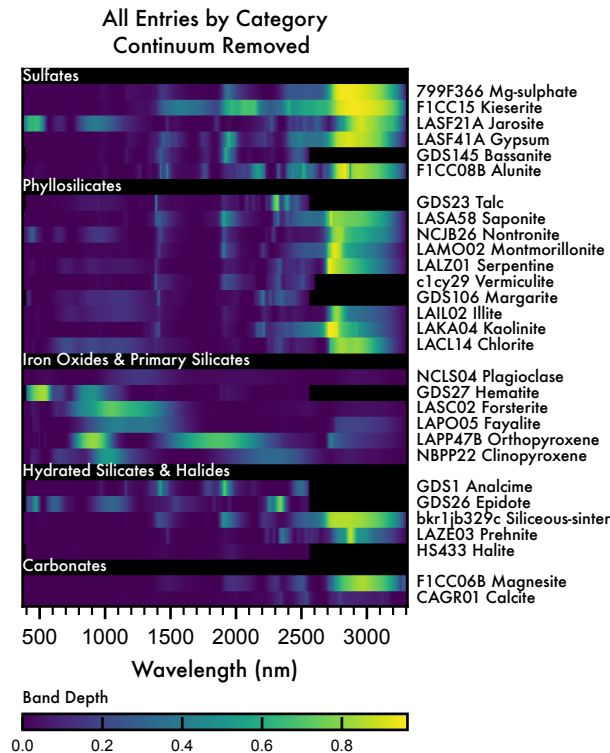


Figure 5 Continuum-removed waterfall plot of high resolution MICA Files spectral library, grouped by Category.

Again, we can see that the waterfall plot represents the band depth information of the `MaterialCollection` in a compact format. Scanning vertically allows for quick identification of common band locations, their depths, widths, and asymmetries.

5 Results

5.1 Simulating the EnfyS Spectral Response

To simulate EnfyS, rather than providing a file with the expected spectral response of each position of the Linear-Variable Filter (LVF, above), we specify the spectral range and resolution and sampling regime, and generate a set of central-wavelengths and bandwidths. We do this with the `InstrumentBuilder` class of the `sptk.instrument` module.

```
[ ]: from sptk.instrument import InstrumentBuilder
```

5.1.1 Signal-to-Noise Ratio

We obtain the expected SNR from the `enfys_inas_mwir_snr.csv` file, derived from the EXM-EN-MTX-ABU-0001_Enfys_Science_Traceability_Matrix_3-0 spreadsheet, computed by P.I. M. Gunn.

We have adapted the `InstrumentBuilder` function to accept a pandas Series mapping wavelength to SNR. The `InstrumentBuilder` then performs interpolation to the simulated wavelength range.

We read in the SNR file:

Here we are going to artificially scale the InAs SNR by a factor of 1/10, to simulate an arbitrary ‘worst-case scenario’.

```
[59]: import numpy as np
      inas_mwir_snr_max = np.nanmax(inas_mwir_snr.to_numpy())
      inas_mwir_snr = inas_mwir_snr / 10
      inas_mwir_snr.min(), inas_mwir_snr.max()
```

```
[59]: (0.0, 67.4371531900000004)
```

```
[13]: import pandas as pd

      enfys_snr = pd.read_csv('../data/instruments/enfys_snr.csv', index_col=0)
```

and get the SNR data for the InAs MWIR sensor,

```
[14]: inas_mwir_snr = enfys_snr['InAs MWIR SNR']
```

and the InGaAs SWIR sensor,

```
[15]: ingaas_swir_snr = enfys_snr['InGaAs SWIR SNR']
```

Here we are going to artificially scale the InAs SNR by a factor of 1/10, to simulate an arbitrary ‘worst-case scenario’.

```
[16]: import numpy as np
      inas_mwir_snr_max = np.nanmax(inas_mwir_snr.to_numpy())
```

```
[ ]: inas_mwir_snr = inas_mwir_snr / 10
```

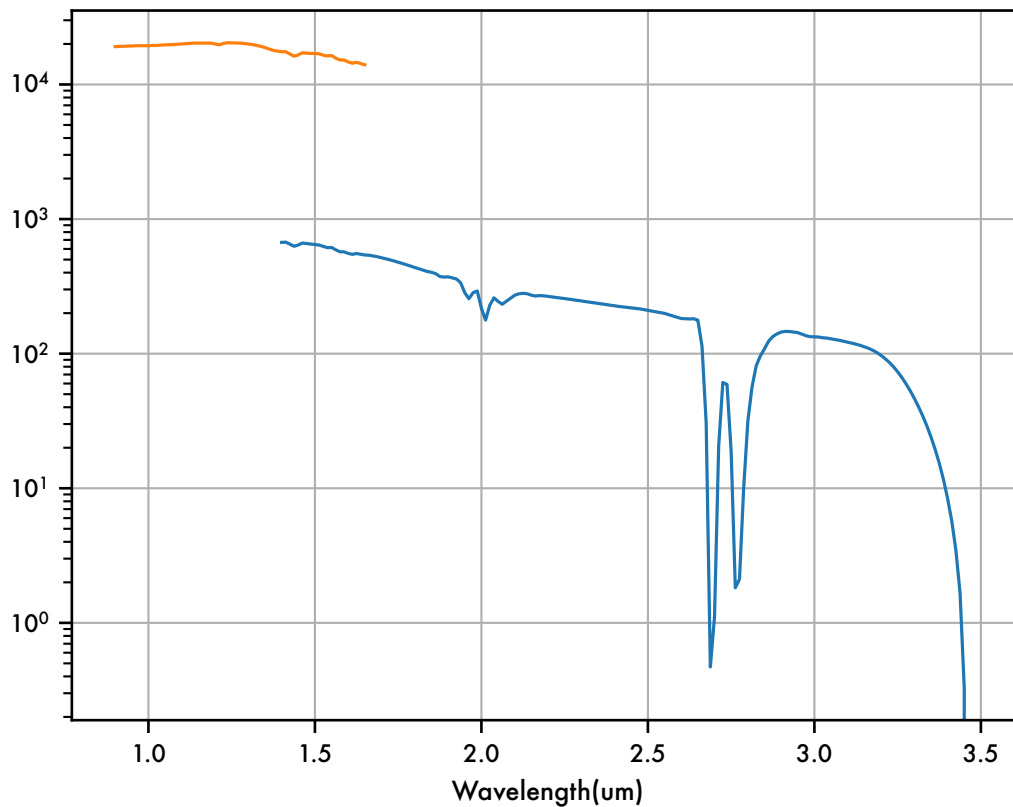
```
[18]: inas_mwir_snr.min(), inas_mwir_snr.max()
```

```
[18]: (0.0, 674.371531900000004)
```

The spectral SNR for each of the SWIR InGaAs and MWIR InAs sensors are:

```
[22]: inas_mwir_snr.plot(logy=True, grid=True)
      ingaas_swir_snr.plot(grid=True)
```

```
[22]: <AxesSubplot: xlabel='Wavelength(um) '>
```



We splice these together, using the higher value SNR where the wavelength ranges overlap:

```
[ ]: # concatenate the inas_mwir_snr and the ingaas_swir, using the highest SNR for overlapping wavelength values
     enfys_inas_mwir_snr = ingaas_swir_snr.combine_first(inas_mwir_snr)
```

We also convert the wavelengths from μm to nm:

```
[ ]: enfys_inas_mwir_snr.index = enfys_inas_mwir_snr.index * 1000
```

for computational convenience, we should set values of $\text{SNR} \leq 1$ to NaN, to interpret these as ‘bad bands’

```
[ ]: # where SNR <=1 to NaN, to interpret these as 'bad bands'
     import numpy as np
     enfys_inas_mwir_snr = enfys_inas_mwir_snr.where(enfys_inas_mwir_snr > 3,
     other=np.nan)
```

```
[ ]: enfys_inas_mwir_snr.index.name = 'wavelength'
```

The continuous spectral SNR for the complete instrument is:

```
[ ]: g = enfys_inas_mwir_snr.plot(logy=True)
      # set the x and y axes
      g.set_xlabel('Wavelength (nm)')
      g.set_ylabel('SNR')
```

5.1.2 Spectral Resolving Power

We also load in new data for the spectral resolving power of the SWIR and MWIR LVFs of Enfys. We merge these datasets, using the SWIR values where the sensor range overlaps.

```
[ ]: # get the max wavelength where ingaas_swir_snr is not nan
ingaas_swir_snr = ingaas_swir_snr[~np.isnan(ingaas_swir_snr)]
max_swir_wavelength = ingaas_swir_snr.index[-1] * 1000 # convert from μm to nm
max_swir_wavelength
# get the enfys_swir_respow data where the wavelength is less than the
↳max_swir_wavelength
enfys_swir_respow = enfys_swir_respow[enfys_swir_respow.index <
↳max_swir_wavelength]
enfys_swir_respow
# get the enfys_mwir_respow where the wavelength is greater than the
↳max_swir_wavelength
enfys_mwir_respow = enfys_mwir_respow[enfys_mwir_respow.index >
↳max_swir_wavelength]
enfys_mwir_respow
# now merge the datasets
enfys_respow = pd.concat([enfys_swir_respow, enfys_mwir_respow])
```

The complete instrument Spectral Resolving Power is:

```
[ ]: g=enfys_respow.plot(xlabel='Wavelength (nm)', ylabel=r'$\lambda/\Delta\lambda$
↳Spectral Resolving Power')
g.legend(['SWIR & MWIR'])
```

We pass this data to the InstrumentBuilder function.

5.1.3 Instrument Build

Now we will pass this to the InstrumentBuilder:

```
[ ]: enfys_builder = InstrumentBuilder(
      instrument_name='enfys_inas_mwir',
      instrument_type='lvf',
      sampling='nyquist',
      resolution = enfys_respow,
      spectral_range=[900, 3101],
      snr = enfys_inas_mwir_snr)
```

```
[ ]: from sptk.instrument import Instrument
```

Now we build the transmission profiles for each LVF position, according to the list of CWLs and FWHMs generated above. These are accessed by reading the `.csv` file produced by the `InstrumentBuilder` procedure.

When generating the profiles, we turn the CWL and FWHM into a profile by assuming a distribution function. For *Enfys* we use the Cauchy distribution, under the guidance of the *Enfys* build team, which behaves similarly to a Gaussian, but with wider ‘wings’. It is parameterised by the full-width-at-half-maximum, as the distribution strictly has no finite variance. We have implemented a `build_cauchy_filter` method for the `Instrument` class.

```
[ ]: enfys = Instrument(
    name = 'enfys_inas_mwir', # filename of the instrument information,
    ↪held in /software/data/instruments/
    project_name = matcol.project_name, # the project directory name,
    ↪hosting outputs in the /spectral_parameter_studies/ directory
    shape = 'cauchy',
    load_existing=False, # if True, load existing instrument data from
    ↪the project directory
    plot_profiles=False, # if true, plot the transmission profiles
    ↪during construction
    export_df=True) # if True, export the instrument transmission
    ↪profiles to the project directory
```

```
[ ]: fig ,ax = enfys.plot_instrument_characteristics()
```

Figure 6 Simulated spectral response and resolution of *Enfys*, based on preliminary expectations of the spectral and radiometric response.

The data shown in figure 3 approximates the expected performance of *Enfys*. Once the instrument is calibrated at the component and assembled levels, this data can be replaced with empirically acquired data.

5.2 Sampling MICA with Enfys

We now make an `Observation` object by sampling the MICA files `MaterialCollection` with the *Enfys* `Instrument`.

```
[ ]: from sptk.observation import Observation
```

```
[ ]: obs = Observation(
    material_collection=matcol,
    instrument = enfys,
    load_existing=False,
    plot_profiles=False,
    export_df=True)
```

We can plot the sampled spectra against the input high-spectra below:

```
[ ]: axes = obs.plot_profiles(scope='categories', groupby='Species', stacked=True,
    ↪hires_under=True)
```

Figure 7 The MICA files as sampled by *Enfys*, with the original high-resolution spectra plotted underneath each.

We can now see where there are deviations of the *Enfys* sampled spectra from the high-resolution source spectra.

For example, illite, plotted below for clarity.

```
[ ]: axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,
    ↪ hires_under=True)
```

Figure 8 Deviation of the *Enfys* sampled spectrum from the high-resolution input spectrum for Illite.

We note that there is an effect by which peaks and troughs in the spectra are reduced. This is due to the long tails of the Cauchy filter profiles. Despite the reduction of absolute band depth, the features of Illite (fig. 8) are still resolvable.

We can produce a continuum-removed waterfall plot to check that the major bands are resolved.

```
[ ]: from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

obs_cr = sla(obs).remove_continuum()
```

```
[ ]: axes = obs_cr.plot_profiles(waterfall=True, scope='all', groupby='Category')
```

Figure 9 Continuum-Removed waterfall plot of the MICA files as sampled by *Enfys*.

Comparing Figure 9 to Figure 5, we can see that the weaker sub-2 μm bands are less well resolved.

We can save this data

```
[ ]: obs_cr.export_main_df()
```

5.3 Adding Synthetic Noise

We have the capability to add synthetic noise to the dataset, by redrawing a sample from the dataset with noise added according to the signal-to-noise ratio specified for the given channel.

We can optionally redraw multiple noisy samples for each entry, to give an indication of the statistical uncertainty, or for example draw a single noisy sample, to produce a single-observation mission-realistic dataset.

First, let's draw a single noisy entry:

```
[ ]: noise_obs = obs.add_noise(1, seed=0)
```

```
[ ]: axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,
    ↪ hires_under=True, ci=False)
```

Figure 10 Noisy resampled example spectrum for Illite, showing 1 noisy sample according to the spectral signal-to-noise ratio illustrated in figure 3.B.

We can also plot the mean observed line, with shading indicating the standard deviation of the noisy data, shown below in figure 7, where the ‘ci’ keyword has been applied (‘ci’: confidence-interval).

```
[ ]: axes = obs.plot_profiles(scope='illite', groupby='Sample ID', stacked=False,
    ↳ hires_under=True, ci=True)
```

Figure 11 Mean of Noisy resampled example spectrum for Illite, showing mean and standard deviation of 100 repeat samples according to the spectral signal-to-noise ratio illustrated in figure 3.B.

Here we find that the standard deviation is almost negligible at the scale of the figure.

Plotting out all of the groups, we again see that the noise, at this scale, is indeed negligible for most entries. Here we opt to plot the complete range of noisy samples, rather than averaging and showing the (very small) standard deviation.

```
[ ]: axes = obs.plot_profiles(scope='categories', groupby='Species', stacked=True,
    ↳ hires_under=True, ci=False)
```

Figure 12 The MICA files as sampled by *Enfys*, with repeat samples drawn according to the expected spectral Signal-to-Noise Ratio, with the original high-resolution spectra plotted underneath each.

The key conclusion to draw from the above figure is that expected noise for *Enfys* does not significantly intrude on the diagnostic features of these type specimen of minerals identified on the Mars surface.

We can illustrate this in greater deal by plotting each entry on a separate figure, with the y-axis clipped to the spectral range of entry. Note that for sample with a small spectral range (i.e. samples that are spectrally flat), this stretching will appear to amplify noise, so the y-axis range must be considered when assessing the noise.

```
[ ]: axes = obs.plot_profiles(scope='samples', groupby='Species', stacked=False,
    ↳ hires_under=True, ci=False)
```

Figure 13 Detailed view of each entry in the spectral library, showing the scale of noise introduced, relative to the features of each spectrum, and the reflectance range.

We can export the dataset:

```
[ ]: obs.export_main_df()
```

5.4 Continuum Removal

We can apply continuum removal, even to the noisy data.

```
[ ]: from sptk.spectral_library_analyser import SpectralLibraryAnalyser as sla

cr_tool = sla(obs)
cr_obs = cr_tool.remove_continuum()
```

```
[ ]: axes = cr_obs.plot_profiles(scope='categories', groupby='Species',
    ↪stacked=True, hires_under=False, ci=False)
```

Figure 14 Continuum-Removed entries of the MICA Files, as sampled by *Enfys*, including synthetic noise.

Again, we can export this:

```
[ ]: cr_obs.export_main_df()
```

5.5 Waterfall Visualisation

For waterfall visualisations, we have the advantage of being able to visualise the noise introduced for each of multiple entries in 2D.

Each bar of the visualisation is composed of a series of rows, with each row as a redrawn noisy sample.

For this, we redraw with 16 repeat samples.

```
[ ]: obs.add_noise(16, seed=0)
```

```
[ ]: axes = obs.plot_profiles(scope='all', groupby='Category', stacked=False,
    ↪waterfall=True, ci=False)
```

Figure 15 Waterfall diagram of the entries of the MICA Files, as sampled by *Enfys*, including synthetic noise. Each entry bar is composed of n repeat noisy samples, such that each bar is composed of n rows, illustrating the noise.

We need to recompute the continuum-removed data on this noisy set.

```
[ ]: cr_tool = sla(obs)
    cr_obs = cr_tool.remove_continuum()
```

```
[ ]: axes = cr_obs.plot_profiles(scope='all', groupby='Category', stacked=False,
    ↪waterfall=True, ci=False)
```

Figure 16 Continuum-Removed waterfall diagram of the entries of the MICA Files, as sampled by *Enfys*, including synthetic noise. Each entry bar is composed of n repeat noisy samples, such that each bar is composed of n rows, illustrating the noise.

```
[ ]: axes = cr_obs.plot_profiles(scope='samples', groupby='Species', stacked=False,
    ↪waterfall=True, ci=False)
```

Figure 17 Detailed waterfall view of each entry in the spectral library, showing the scale of noise introduced, relative to the features of each spectrum, and the reflectance range. Each entry bar is composed of n repeat noisy samples, such that each bar is composed of n rows, illustrating the noise.

5.5.1 Validating the Noise

We can check that our noise is correct by aggregating the repeat samples to give the mean and standard deviation for each entry and each spectral channel (wavelength).

We can then compare this to the spectral SNR.

```
[ ]: signal = obs.main_df.groupby('Root Data ID').mean()
     noise = obs.main_df.groupby('Root Data ID').std()
     snr = signal/noise

[ ]: snr.mean()

[ ]: # plot the SNR
     import seaborn as sns
     # plot with log scale y axis
     g = sns.lineplot(data=snr.mean(), label='Mean/Standard Deviation of Sampled_
           ↳Noisy Data', markers=True)
     g.set(yscale="log")

     # get instrument snr
     sns.lineplot(data=obs.instrument.main_df, x='cwl', y='snr', label='InGaAs +_
           ↳InAs SNR', linestyle='--', markers=True)

     g.set_xlabel('Wavelength (nm)')
     g.set_ylabel('SNR')
```

6 Discriminatory Evaluation with Spectral Angle Mapper

To evaluate the ability of Enfys to discriminate between the materials of the MICA files set, we will use the Spectral Angle to measure the distance between a given noisy spectrum and each clean entry of the MICA files. We expect the smallest Spectral Angle to occur for the matching material.

By running the evaluation on N-repeat noisy samples, we can estimate the propagation of the noise through the Spectral Angle. We can then compare this noise to the difference in Spectral Angle between the match and the next-nearest match, to understand the repeatability.

The Spectral Angle computation is closed-form, and can be formulated in a Linear Algebra representation that scales well with increasing sizes of spectral libraries.

Overview: - define the Spectral Angle evaluation function - run this analysis on each material in the collection

6.1 Spectral Angle Evaluation

Inputs: - Test material name - Test material noisy data - MICA files clean data

Outputs: - Spectral Angle for each MICA entry, and for each repeat noisy sample, order in increasing angle - Plot of Spectral Angle results, showing score for each material, and the noisy spectrum

vs the top 3 matches - Plot of mean Spectral Angle result, with errorbars, to show if final material detection is statistically significant

First, let's get a new noisy dataset with 100 repeat noisy samples for each material.

```
[ ]: noise_obs = obs.add_noise(128, seed=0)
```

Now we take a material from the data set, and get the noisy entries for that material, and the clean dataset for the entire MICA files.

```
[ ]: # getting the material entry to test
materials = obs.main_df['Root Data ID'].unique()
test_material = materials[2]
test_material
```

Getting the noisy dataset for this material:

```
[ ]: noisy_data = obs.get_refl_df(root_data_id=test_material).to_numpy()
noisy_data.shape
# set all nan values to 0
noisy_data = np.where(~np.isfinite(noisy_data), 0, noisy_data)
```

getting the clean data for all materials in the MICA set:

```
[ ]: clean_data = obs.noiseless_df[obs.wvls].to_numpy()
clean_data.shape
# set all nan values to 0
clean_data = np.where(~np.isfinite(clean_data), 0, clean_data)
```

Computing the Spectral Angle between the each noisy data entry and the MICA set.

The Spectral Angle is defined between the noisy spectrum $\tilde{R}_i[\lambda]$ of material i in the set of MICA file materials ($\mathcal{M}$) and the clean spectrum $R_j[\lambda]$, where $j \in \mathcal{M}$.

The Spectral Angle θ is:

$$\theta(i, j) = \arccos \left(\frac{\sum_{\lambda} \tilde{R}_i[\lambda] R_j[\lambda]}{(\sum_{\lambda} \tilde{R}_i[\lambda]^2)^{1/2} (\sum_{\lambda} R_j[\lambda]^2)^{1/2}} \right)$$

We can formulate the numerator $\sum_{\lambda} \tilde{R}_i[\lambda] R_j[\lambda]$ as

```
[ ]: numerator = np.inner(noisy_data, clean_data)
numerator.shape
```

and the denominator parts as:

```
[ ]: denominator = np.outer(np.sqrt(np.sum(noisy_data**2, axis=1)), np.sqrt(np.
    ↳sum(clean_data**2, axis=1)))
denominator.shape
```

we can then combine these to compute the Spectral Angle as:

```
[ ]: sa = np.arccos(numerator / denominator)
```

```
[ ]: sa.shape
```

that we can package into a DataFrame:

```
[ ]: sa_df = pd.DataFrame(sa, index=obs.get_refl_df(root_data_id=test_material).  
    ↪index, columns=obs.noiseless_df.index)
```

```
[ ]: sa_df
```

To visualise, first let's take the average Spectral Angle across all of the observations, and also compute the standard deviation.

```
[ ]: # Now let's order the sa_df DataFrame columns in ascending mean value order  
sa_df = sa_df.loc[:, sa_df.mean(axis=0).sort_values().index]
```

```
[ ]: sa_ave = sa_df.mean(axis=0)  
sa_std = sa_df.std(axis=0)
```

Now let's order the sa_df DataFrame columns in ascending mean value order

```
[ ]: sa_ave.sort_values().index
```

Now collecting all of these functions in a set of callable functions, such that we can run this analysis on each entry.

Here we also compute a 'baseline' Spectral Angle, between each clean entry and each other clean entry.

```
[ ]: def spectral_angle(data_a, data_b):  
    # compute spectral angle  
    numerator = np.inner(data_a, data_b)  
    denominator = np.outer(np.sqrt(np.sum(data_a**2, axis=1)), np.sqrt(np.  
    ↪sum(data_b**2, axis=1)))  
    sa = np.arccos(numerator / denominator)  
    return sa  
  
def compute_spectral_angle(obs, material_name):  
    # get noisy data  
    noisy_data = obs.get_refl_df(root_data_id=material_name).to_numpy()  
    bad_bands = ~np.isfinite(noisy_data[0,:])  
    noisy_data = np.where(bad_bands, 0, noisy_data) # set all nan values to 0  
  
    # get clean data  
    clean_data = obs.noiseless_df[obs.wvls].to_numpy()  
    clean_data = np.where(bad_bands, 0, clean_data) # 0 the clean data where  
    ↪the noisy data is NaN  
  
    sa_noisy = spectral_angle(noisy_data, clean_data)
```

```

# get baseline Spectral Angle
baseline_data = obs.noiseless_df.loc[material_name][obs.wvls].to_numpy()
baseline_data = np.expand_dims(baseline_data, axis=0).astype(np.float64)
baseline_data = np.where(bad_bands, 0, baseline_data) # 0 the clean data
↳where the noisy data is NaN
sa_baseline = spectral_angle(baseline_data, clean_data)

# package in DF
sa_df = pd.DataFrame(sa_noisy, index=obs.
↳get_refl_df(root_data_id=material_name).index, columns=obs.noiseless_df.
↳index)
sa_bl = pd.Series(sa_baseline.squeeze(), index=obs.noiseless_df.index)
# set sa_bl for index of material_name to 0
sa_bl.loc[material_name] = 0.0

# order DF columns in baseline value order
sa_df = sa_df.loc[:, sa_bl.sort_values().index]
sa_bl = sa_bl.loc[sa_bl.sort_values().index]

return sa_df, sa_bl

```

```
[ ]: sa_df, sa_bl = compute_spectral_angle(obs, test_material)
```

```
[ ]: from matplotlib import pyplot as plt
```

```

[ ]: def plot_spectral_angle_noisy_map(sa_df, material_name):
    fig, ax = plt.subplots()

    im = ax.imshow(sa_df.to_numpy().T,
                    cmap='inferno_r',
                    interpolation='none',
                    vmin=0, vmax=1,
                    aspect='auto')
    # add color bar called 'Spectral Angle'
    ax.set_xlabel('N-Repeat')
    ax.set_ylabel('Entry Ranked by Asc. Baseline Spectral Angle')
    cbar = ax.figure.colorbar(im, ax=ax, location='bottom')
    cbar.ax.set_xlabel('Spectral Angle')
    # use index for axis labels
    # set every tickmark to the corresponding index
    yticks=ax.set_yticks(np.arange(len(sa_df.columns))+0.5)
    ylbls=ax.set_yticklabels(sa_df.columns.to_list(), rotation=0)

    # add vertical gridlines
    ax.grid(which='major', axis='y', color='white', linestyle='-', linewidth=0.
↳2)

```

```

# add a top x-axis called Match Rank, with ticks aligned to the right
ax2 = ax.twinx()
ax2.set_ylim(ax.get_ylim())
ax2.set_yticks(np.arange(len(sa_df.columns))+0.5)
ax2.set_yticklabels(np.arange(1, len(sa_df.columns)+1),
horizontalalignment='left', verticalalignment='bottom', fontsize=7)
ax2.set_ylabel('Match Rank')

fig.suptitle(f'{material_name} Spectral Angle Match')
fig.tight_layout()

return fig, ax

```

```
[ ]: plot_spectral_angle_noisy_map(sa_df, test_material)
```

```
[ ]: sa_df
```

```
[ ]: sa_bl
```

```

[ ]: # alternatively, we can plot the Spectral Angle score against the entry
def plot_mean_spectral_angle(sa_df, sa_bl, material_name):
    fig, ax = plt.subplots()
    ax.plot(sa_df.mean(axis=0), marker='o', label='Ave. ± Std. Dev.')
    ax.plot(sa_bl, marker='o', label='Baseline')
    ax.legend(loc='upper left')
    # set ylimits
    ax.set_ylim(0, 1)
    # add error bars
    ax.errorbar(np.arange(len(sa_df.columns)), sa_df.mean(axis=0), yerr=sa_df.
std(axis=0), fmt='o', capsize=5)
    ax.fill_between(np.arange(len(sa_df.columns)), sa_df.mean(axis=0) - sa_df.
std(axis=0), sa_df.mean(axis=0) + sa_df.std(axis=0), alpha=0.2)
    ax.set_ylabel('Mean Spectral Angle  $\pm \sigma$ ')
    ax.set_xlabel('Entry Ranked by Asc. Mean Spectral Angle')
    xbls=ax.set_xticklabels(sa_df.columns.to_list(), rotation=90)

    # add vertical gridlines
    ax.grid(which='major', axis='x', color='gray', linestyle='-', linewidth=0.2)

    # add a top x-axis called Match Rank, with ticks aligned to the right
    ax2 = ax.twinx()
    ax2.set_xlim(ax.get_xlim())
    ax2.set_xticks(np.arange(len(sa_df.columns)))
    ax2.set_xticklabels(np.arange(1, len(sa_df.columns)+1),
horizontalalignment='right', verticalalignment='center', fontsize=7)
    ax2.set_xlabel(f'{material_name} Match Rank')

```

```
return fig, ax
```

```
[ ]: plot_mean_spectral_angle(sa_df, sa_bl, test_material)
```

Finally, let's plot the top 3 results in a spectral library analyser plot.

```
[ ]: top3 = sa_df.mean(axis=0).sort_values().index[:3].to_list()
top3
```

```
[ ]: ax = obs.plot_profiles(scope=top3, hires_under=True, stacked=True,
    ↳groupby='Sample ID', ci=False)
# drop title
# get figure from ax
fig = ax[0].figure
# drop title from fig
fig.suptitle('')
```

```
[ ]: for material in materials:
    print(f'Evaluating Spectral Angle for {material}')
    sa_df, sa_bl = compute_spectral_angle(obs, material)

    fig, ax = plot_spectral_angle_noisy_map(sa_df, material)
    # fig.savefig(f'../reports/figures/
    ↳{PROJECT_NAME}_{material}_spectral_angle_map.png', dpi=300,
    ↳bbox_inches='tight')
    # plt.close(fig)

    fig, ax = plot_mean_spectral_angle(sa_df, sa_bl, material)
    # fig.savefig(f'../reports/figures/
    ↳{PROJECT_NAME}_{material}_mean_spectral_angle.png', dpi=300,
    ↳bbox_inches='tight')
    # plt.close(fig)

    top3 = sa_df.mean(axis=0).sort_values().index[:3].to_list()
    ax = obs.plot_profiles(scope=top3, hires_under=True, stacked=True,
    ↳groupby='Sample ID', ci=False)
    # drop title
    # get figure from ax
    fig = ax[0].figure
    # drop title from fig
    fig.suptitle(f'Top 3 Matches for {material} w/ Noise')

    top3 = sa_df.mean(axis=0).sort_values().index[:3].to_list()
    ax = obs.plot_profiles(scope=top3, hires_under=True, stacked=True,
    ↳groupby='Sample ID', ci=True)
    # drop title
```

```

# get figure from ax
fig = ax[0].figure
# drop title from fig
fig.suptitle(f'Top 3 Matches for {material} Ave.')

```

6.2 Spectral Angle Mapper

TODO: perform spectral angle mapper evaluation of each spectrum against the other spectra of the set.

First, we prepare numpy arrays of the noisy and clean data.

```

[ ]: noise_obs = obs.add_noise(1, seed=0)

[ ]: noisy_data = obs.get_refl_df().to_numpy()

[ ]: clean_obs = Observation(
    material_collection=matcol,
    instrument = enfys,
    load_existing=False,
    plot_profiles=False,
    export_df=True)
clean_data = clean_obs.get_refl_df().to_numpy()

```

Need to remove NaN entries from noisy and clean data.

Now we use these 2 datasets to compute the Spectral Angle Mapper between each entry and each other entry

```

[ ]: # sum = np.dot(clean_data, noisy_data.T)
sum = np.nansum(clean_data * noisy_data, axis=1)
sqrt = np.sqrt(np.nansum(clean_data*noisy_data, axis=1))
sqrt_mat = np.outer(sqrt, sqrt)
sam = np.arccos(sum / sqrt_mat)

[ ]: sam_df = pd.DataFrame(sam, index=obs.main_df.index, columns=clean_obs.main_df.
    ↪index)

[ ]: from matplotlib import pyplot as plt

[ ]: fig, ax = plt.subplots()

im = ax.imshow(sam_df.to_numpy(),
               cmap='inferno_r',
               interpolation='none',
               aspect='auto')
# add color bar called 'Spectral Angle'
cbar = ax.figure.colorbar(im, ax=ax)
cbar.ax.set_ylabel('Spectral Angle')

```

```

# use index for axis labels
# set every tickmark to the corresponding index
yticks=ax.set_yticks(np.arange(len(obs.noiseless_df.index))+0.5)
ylbls=ax.set_yticklabels(sam_df.index.to_list(), rotation=0)
xticks=ax.set_xticks(np.arange(len(obs.noiseless_df.index))+0.5)
xlbls=ax.set_xticklabels(sam_df.index.to_list(), rotation=90)

ax.grid(True)

```

Lets now plot for each entry an ordered list of the match results. For each mineral, this will show which minerals are most uniquely discriminated.

```

[ ]: def plot_single_entry_sam(sam_results, spectrum_name):
    fig, ax = plt.subplots()
    im = ax.imshow(sam_results.to_numpy().reshape(1, -1).T,
                    cmap='viridis_r',
                    interpolation='none',
                    vmin=0, vmax=1,
                    aspect='auto')

    # add color bar called 'Spectral Angle'
    cbar = ax.figure.colorbar(im, ax=ax)
    cbar.ax.set_ylabel('Spectral Angle')
    cbar.ax.invert_yaxis()

    # use index for axis labels
    # set every tickmark to the corresponding index
    yticks=ax.set_yticks(np.arange(len(sam_results.index)))
    ylbls=ax.set_yticklabels(sam_results.index.to_list(), rotation=0)
    ax.set_title(f'Spectral Angle Map - {spectrum_name}')

    # plot the sample against the library
    # fig, ax = plt.subplots()
    # ax.plot(obs.wvls, x, label='Sample', color='black')
    # ax.plot(obs.wvls, lib.loc[sam_results.index[0], obs.wvls],
    ↪label=sam_results.index[0], color='green')
    # ax.plot(obs.wvls, lib.loc[sam_results.index[1], obs.wvls],
    ↪label=sam_results.index[1], color='orange')
    # ax.plot(obs.wvls, lib.loc[sam_results.index[2], obs.wvls],
    ↪label=sam_results.index[2], color='red')
    # ax.legend()

[ ]: # get a noiseless sample entry to try
for sample, data in obs.main_df.iterrows():
    sample_name = sample
    noisy_sample = data[obs.wvls].to_numpy(dtype=np.float64).flatten()

    # sum = np.dot(clean_data, noisy_sample)
    sum = np.nansum(clean_data * noisy_sample, axis=1)

```

```
sqrt_sum = np.sqrt(np.nansum(clean_data**2, axis=1)) * np.sqrt(np.  
↪nansum(noisy_sample**2))  
sam = np.arccos(sum/sqrt_sum)  
  
sam_results = pd.Series(data=sam, index=clean_obs.main_df.index)  
sam_results.sort_values(ascending=True, inplace=True)  
  
plot_single_entry_sam(sam_results, sample_name)
```

7 Discussion

Notes on what we found above, perhaps with some more analysis.

8 Conclusions

Concluding remarks and summary of findings.

9 References